

Guardrails

A hands-on course in building agents by building their harness

Lesson 6

The box limits reach, not whether

Inside Lesson 5's box, the agent still runs whatever the model asks, as soon as it asks, for as long as it keeps asking.

- `rm -rf` on the mounted folder: your real project, gone
- Stuck retrying one command: it goes on until you notice the bill

The model proposes; the harness decides

Guardrails are rules about what the harness is allowed to do, checked before it does it: which commands may run, when a person gets a say, when a run has gone on long enough.

Built on control flow, which decides whether the next step happens at all:

- It turns a tool request into a command being run
- It turns the result into another model call
- A guardrail is a decision at one of those two points, so it lives there

A few checks, where the loop already passes

- **A gate before each tool:** *allow* it, *ask* a person first, or *deny* it
- **A refusal the model can read:** in the result's slot, marked as an error, saying why
- **Limits before each call:** count the steps and tokens; past the limit, stop
- **An interrupt:** ESC stops it at once, thinking, saying or acting, tells the model, and hands the run back

The box comes first

- Guardrails don't change what the model asks for, only whether the harness goes along with it
- They're only as good as the rules: a check is code reading a command
- A command can say the same thing many ways
- The guard keeps out what you'd never want; the box limits what gets through

The policy and the gate

Lesson 5's `quark.py` plus 62 lines. At the top of `# — control flow —`
(abridged):

```
MAX_STEPS, MAX_TOKENS = 20, 200_000
SAFE = {"ls", "cat", "head", "tail", "wc", "grep", "pwd", "date", "echo", "du", ...}
DENY = re.compile(r"\bsudo\b|rm\s+-\w*[rf]|mkfs|git\s+push|(curl|wget).*\|\s*(ba)?sh|\s.env\b")
def guard(cmd):
    if DENY.search(cmd): return "blocked by policy"
    if not re.search(r"[;<>$`\n()]", cmd) and all(... in SAFE ...): return None
    answer = read(f"allow `{cmd}`? [y/N] ")
    return None if answer.lower() == "y" else "the person said no" + ...
```

`guard()`: deny, allow, or ask

It returns `None` if the command may run, or the reason it may not:

1. Matches `DENY`: refused outright, and no one is asked
2. Only safe programs: it runs. No `;` `&` `<` `>` `$`, backtick, newline or `(`, and every piece between pipes starts with a `SAFE` program
3. Anything else goes to the person with Lesson 2's `read()`; a `y` lets it run
4. Any other answer is a no, and what they typed goes back to the model

Limits, checked before every call

At the top of the loop, and where each call is counted (abridged):

```
if steps >= MAX_STEPS or spent >= MAX_TOKENS:
    print(f"[stopped: {steps} steps, {spent} tokens]")
    if not chat or (input := read("\n> ")) == "/q": break
    add(working_memory, {"role": "user", "content": input})
    steps, spent = 0, 0
    continue
...
with listening():
    response = call(unless_esc, ..., messages=working_memory)
    output = response.content
    steps += 1
    spent += response.usage.input_tokens + response.usage.output_tokens + ...
```

In a one-shot run, a limit is the end. In a chat it hands back to you, and the counts start again after each new input.

A refusal goes where the result would have

In the output loop, between printing the command and running it in the box (abridged):

```
print(f"$ {block.input['cmd']}")
if ESC.is_set():
    input.append(..., "content": "[your doing never reached the world]")
    continue
if (no := guard(block.input["cmd"])):
    print(f"[{no}]")
    input.append({"type": "tool_result", ..., "content": no, "is_error": True})
    continue
```

- `continue` skips the run: the command never starts, and the model's next turn still makes sense
- The `ESC` check above the gate: once you've pressed ESC, nothing else starts

The interrupt: listening for ESC

ESC is input from a person, so in `# — input —`, after `read()` (abridged):

```
ESC = threading.Event()
SAYING = "[other self interrupted what you were saying – acknowledge]"
DOING = "[other self interrupted what you were doing – acknowledge]"
def watch(stop):
    ...
@contextlib.contextmanager
def listening():
    ESC.clear()
    if not sys.stdin.isatty(): yield; return
    ...
```

- `watch()` reads one key at a time in a thread; an ESC on its own sets `ESC`
- quark listens only while it thinks, says or acts, never while `read()` waits for you

Stopping what it says: the next piece ends it

One line in `call()`, `unless_esc` in place of `show`, and a check right after the call (abridged: end-of-line comments cut):

```
        for event in stream:
            if each(event): return stream.current_message_snapshot
def unless_esc(event):
    if ESC.is_set(): return True
    show(event)
...
    if response.stop_reason is None:
        print()
        add(working_memory, {"role": "user", "content": SAYING})
        continue
```

Stopping what it does: `kill -9 -1` in the box

The command now starts with `Popen`, and quark waits a tenth of a second at a time (abridged):

```
with listening():
    doing = subprocess.Popen(["docker", "exec", box, ..., block.input["cmd"]], ...)
    while True:
        try: done = ...(doing.communicate(timeout=0.1)[0]); break
        except subprocess.TimeoutExpired:
            if ESC.is_set(): subprocess.run([..., "kill -9 -1"], ...)
    done.returncode = doing.returncode
    if ESC.is_set(): done.stdout = "[your doing stopped before done]"
```

- Every command in the box stops; the box itself survives
- What it had printed is dropped; the model gets `DOING`

Safe commands run; anything else waits for you

`wc` is on the safe list, so nothing was asked:

```
$ wc -l notes.txt
3 notes.txt

notes.txt has 3 lines.
```

`rm` isn't, so the harness stopped at the prompt (I piped in `y` ; shortened):

```
$ rm a.log b.log && ls -A
allow `rm a.log b.log && ls -A`? [y/N] .env
.quark
notes.txt
```

A no with a reason; a deny that asks no one

Answers piped in: `no`, put it in a folder called `drafts`, then `y` (shortened):

```
$ touch notes2.txt && ls -l notes2.txt  
allow `touch notes2.txt && ls -l notes2.txt`? [y/N] [the person said no: no, put it in a folder called drafts]  
$ mkdir -p drafts && touch drafts/notes2.txt && ls -l drafts/notes2.txt
```

Then: print `.env`, then `rm -rf .`:

```
$ cat .env  
[blocked by policy]
```

A limit stops the run before the next call

`MAX_STEPS` lowered to 2 in a copy, asking for three commands, one at a time:

```
$ date
Tue Oct 6 20:31:57 UTC 2026

$ pwd
/tmp/g6

[stopped: 2 steps, 15041 tokens]
```

`MAX_TOKENS` works the same way, but at 200,000 you'll rarely reach it.

ESC while it acts: the count stops partway

Count to ten, one a second, **y**, then ESC four and a half seconds later (shortened):

```
$ for i in 1 2 3 4 5 6 7 8 9 10; do echo $i; sleep 1; done  
allow `for i in 1 2 3 4 5 6 7 8 9 10; do echo $i; sleep 1; done`? [y/N] y  
[your doing stopped before done]
```

You interrupted the count, so it stopped before it finished. The command never returned any output, so I can't say how far it got.

ESC while it says: the fable stops mid-sentence

ESC a second after its first full stop (shortened):

```
> Tell me the fable of the tortoise and the hare in about 400 words  
# The Tortoise and the Hare  
...  
I see I was cut off mid-story. Here is the whole fable.  
  
**The Tortoise and the Hare**  
...
```

It stopped at "...a tortoise, who had been plodding past with a leaf on his back".
None of it was kept, so the model told the whole fable again from the top.

ESC while it thinks: back to the request

ESC 0.6 seconds after asking, before it said a word. The blank line is the whole stopped response (shortened):

```
> Make a file called hello.txt containing the word hello.  
  
$ echo hello > hello.txt && cat hello.txt  
allow `echo hello > hello.txt && cat hello.txt`? [y/N] /q  
[the person said no]
```

- Nothing reached the folder: there's no `hello.txt`
- The model is told it was interrupted, not what it was in the middle of
- Keeping what was cut short is Lesson 8's job

Going further: what else guardrails can be

- **How a command is judged:** text patterns, a shell parser, per-tool rules, or a second model call
- **What the answers are:** allow once, for the session, always, or with these arguments
- **Who gets asked:** the terminal, someone on Slack or Telegram, a second agent, or nobody
- **What counts as too much:** steps, tokens, dollars, wall-clock time, or repeating
- **How a run is interrupted:** a key, a message mid-run, a kill switch

As a product alone: **Open Policy Agent, NeMo Guardrails, Guardrails AI.**

`guardrails.py`: finer answers, more limits

Built on Lesson 3's `control_flow.py`, with no system prompt:

- **Jev decides what only reads**, in place of a hand-written list
- **Judged per piece**: `cd /tmp && rm x` is judged by `cd` *and* `rm`
- **Always**: `a` at the prompt adds those programs to `ALLOWED` for the rest of the run
- **A spending limit** in dollars, from each response's `usage`
- **A repeat limit**: the same command `MAX_REPEATS` times stops the run

Asking Jev: read, write, delete or other?

DENIED wins first; then (abridged):

```
def kind(cmd):  
    try:  
        answer = jev.system_one({"command": cmd}, KIND).answers["kind"]  
        return answer.choice, answer.confidence  
    except Exception as e:  
        return f"no answer ({type(e).__name__})", 0.0
```

```
choice, confidence = kind(cmd)  
if choice == "read" and confidence >= SURE:  
    print(f"[Jev: read, {confidence:.2f}]")  
    return "allow", ""
```

- Anything else, or no answer, asks the person, with Jev's answer as the reason

Two questions covered five commands

I answered `a` to `mkdir` and to the first `touch` (shortened):

```
$ mkdir /tmp/gdemo
allow `mkdir /tmp/gdemo`? (Jev: write, 0.99) [y]es, [a]lways, or say why not:
$ cat .env
[blocked by policy: secrets files are off limits]
$ touch /tmp/gdemo/a.txt
allow `touch /tmp/gdemo/a.txt`? (Jev: write, 1.00) [y]es, [a]lways, or say why not:
$ touch /tmp/gdemo/b.txt

$ ls /tmp/gdemo
[Jev: read, 1.00]
a.txt
b.txt
```

The rule: decide before the harness does it

The rule: check each tool request where control flow turns it into a command, and the run where it goes around again. Say no in a way the model can read, and give the person a way in.

What guardrails never do, from inside control flow:

- **Model interface:** one line, so a response can be stopped; otherwise as before
- **Output:** an allowed command runs in the box exactly as before
- **Context:** a refusal is an ordinary tool result in the result's slot
- **Input:** Lesson 2's `read()`, plus a watcher that hears ESC while quark works

What's missing: you can stop a run, not see one

- How long did each step take?
- How many tokens, and what did that cost?
- Which command did the guard refuse, and which did the box kill?
- The episode has every message, but it's written for the model: no timings, no token counts, no exit codes

Next: Observability

Lesson 7